CLOUD COMPUTING MINI PROJECT

# ShopScale

## E-commerce under real-world cloud workloads

Build an online store, simulate traffic bursts, then measure AWS scaling, SLA, recovery and bot detection.



Online store



AWS scaling



Repeatable evidence

[Names] · [Date]

PROBLEM AND SOLUTION

# Success means a working store plus repeatable evidence

## The problem

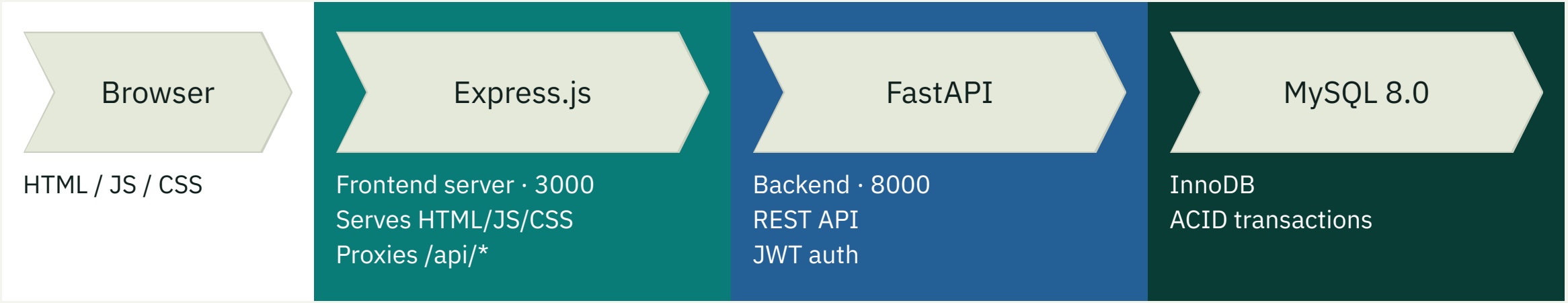
- Traffic spikes during promotions
- Slow APIs hurt trust
- Uncontrolled automation distorts metrics
- Teams need proof that scaling and recovery work

## Our solution

- Working store: catalog, cart, inventory, simulated checkout
- Labelled test traffic
- AWS monitoring, balancing and scaling
- Evaluation of SLA, TTR and detection quality

ARCHITECTURE

# Three design decisions keep the store correct under load




**01**

## Async MySQL driver

The API never blocks under load.

**02**

## Express proxy

One origin, no CORS complexity.

**03**

## Checkout row locks

`SELECT ... FOR UPDATE` prevents overselling.

SCOPE AND STACK

# A focused stack delivers the MVP; stretch goals stay optional

| Layer | Technology |
| --- | --- |
| Frontend | HTML5, vanilla JavaScript, Tailwind CSS |
| Server | Node.js, Express.js |
| Backend | Python, FastAPI, Pydantic v2 |
| Database | MySQL 8.0, SQLAlchemy async, Alembic |
| Testing | pytest, Playwright, Locust |
| Packaging | Docker Compose |
| Cloud | AWS: EC2, ALB, Auto Scaling, RDS, ECR, CloudWatch |

## Must-have MVP

Catalog and search, login, cart, inventory, orders, order history, admin controls.

## Stretch goals

WAF, Prometheus and Grafana, automated recovery, Terraform, Jenkins.

AI anomaly detection as future work.

EVALUATION DESIGN

# Six controlled experiments will produce the evidence

### Playwright journeys

Browse → synthetic-user login → cart → checkout → assert order

### Locust profiles

Baseline, ramp, burst, soak

### Bot experiments

Labelled bots, rate rules, precision and recall, false positives

| Experiment | Evidence to record |
|---|---|
| Baseline | P50, P95, P99, RPS, error rate |
| Ramp | Saturation point |
| Burst | Scale delay, errors, recovery |
| Scale comparison | Latency, throughput, instances, cost |
| Bot controls | Precision, recall, false positives |
| Failure recovery | Detection to TTR |

**Guardrails** · Cap concurrency · Fixed duration · Synthetic accounts · No real payments · Test only our own systems

VALIDATION, NOT RESULTS

# Our SLA targets are hypotheses until experiments validate them

**PROPOSED TARGETS (NOT MEASURED RESULTS)**

| Measure | Hypothesis |
|---|---|
| Availability | At least 99.5% in the test window<br>Proposed target, to be validated by experiments |
| P95 API latency | Under 500 ms at baseline<br>Proposed target, to be validated by experiments |
| Error rate | Under 1% at normal load<br>Proposed target, to be validated by experiments |
| Scale-out | CPU above 70% for 3 minutes; initial hypothesis<br>Proposed target, to be validated by experiments |
| TTR | Under 2 minutes for the chosen failure scenario<br>Proposed target, to be validated by experiments |

TTR workflow

1. Detect
2. Diagnose
3. Recover
4. Verify

$$\mathrm{TTR} = t_{\text{restoration verified}} - t_{\text{incident start}}$$

Results to be recorded

DELIVERY PLAN

# A four-week plan with cost and risk controls built in

| Week 1 | Week 2 | Week 3 | Week 4 |
| --- | --- | --- | --- |
| MVP | Containers and workload | AWS and observability | Experiments and delivery |

## Cost and security controls

AWS Budget alerts first, minimum capacity, tear down test resources, IAM least privilege, private database.

## Key risks

Scope creep → protect the MVP
Integration conflicts → API contract
Race conditions → DB transactions
False bot positives → labelled tests

**Two-week fallback** · Local MVP, Docker, a repeatable load test and one small AWS deployment.

TWO-TEAMMATE WORKFLOW

# Shared rules keep two teammates from breaking each other's work

GitHub Issues and Projects




Branch naming

`type/issue-number-short-name`

Commit style

`type: summary`

PR checklist

- Tests pass
- Teammate review
- Docs updated
- No secrets

AGENTS.md protects shared code. COMMITS.md and UPDATES.md log every change. Squash merge only.

CURRENT STATUS

# The skeleton is underway; backend work starts next

| M0 | M1 | M2 | M3 | M4 | M5 |
| --- | --- | --- | --- | --- | --- |
| Skeleton | Backend | Database | Frontend | Tests and CI | Docker |
| In progress | Not started | Not started | Not started | Not started | Not started |

## M0 skeleton

Repository structure, README, CONTRIBUTING, issue templates, AGENTS.md, COMMITS.md, UPDATES.md, ADR for MySQL merged via PR #57.

## Next slice

**01**

Backend `/health`

**02**

Database models and seed

**03**

Product API

**04**

Product listing page

SHOPSCALE

# Build the store. Stress the system. Prove the results.

Success is a working e-commerce flow, a controlled experiment, honest measurements and a cloud design both teammates can explain.

All SLA targets are hypotheses until validated.

---

Questions?